

Oprogramowanie Systemowe

Warstwa I/O

Block devices vs character devices

- Kernel nie musi udostępniać skomplikowanego podsystemu dedykowanemu obsłudze urządzeń strumieniowych, jak ma to miejsce w przypadku urządzeń blokowych.
- Sektor – fundamentalna, najmniejsza adresowalna jednostka w urządzeniach blokowych (rozmiar jest potęgą 2, najczęściej spotykane jest 512 bajtów). Rozmiar sektora jest fizyczną właściwością urządzenia. Możliwym jest dokonywanie przez urządzenie operacji na większej ilości sektorów jednocześnie.
- Blok – programowo ustalona stała porcja danych. Kernel podczas operacji na dysku twardym operuje na blokach, których rozmiar musi być wielokrotnością sektora. Dodatkowo rozmiar bloku nie może być większy niż wielkość strony pamięci. Standardowe rozmiary bloków to 1,2 bądź 8 sektorów.

Deskryptor bufora

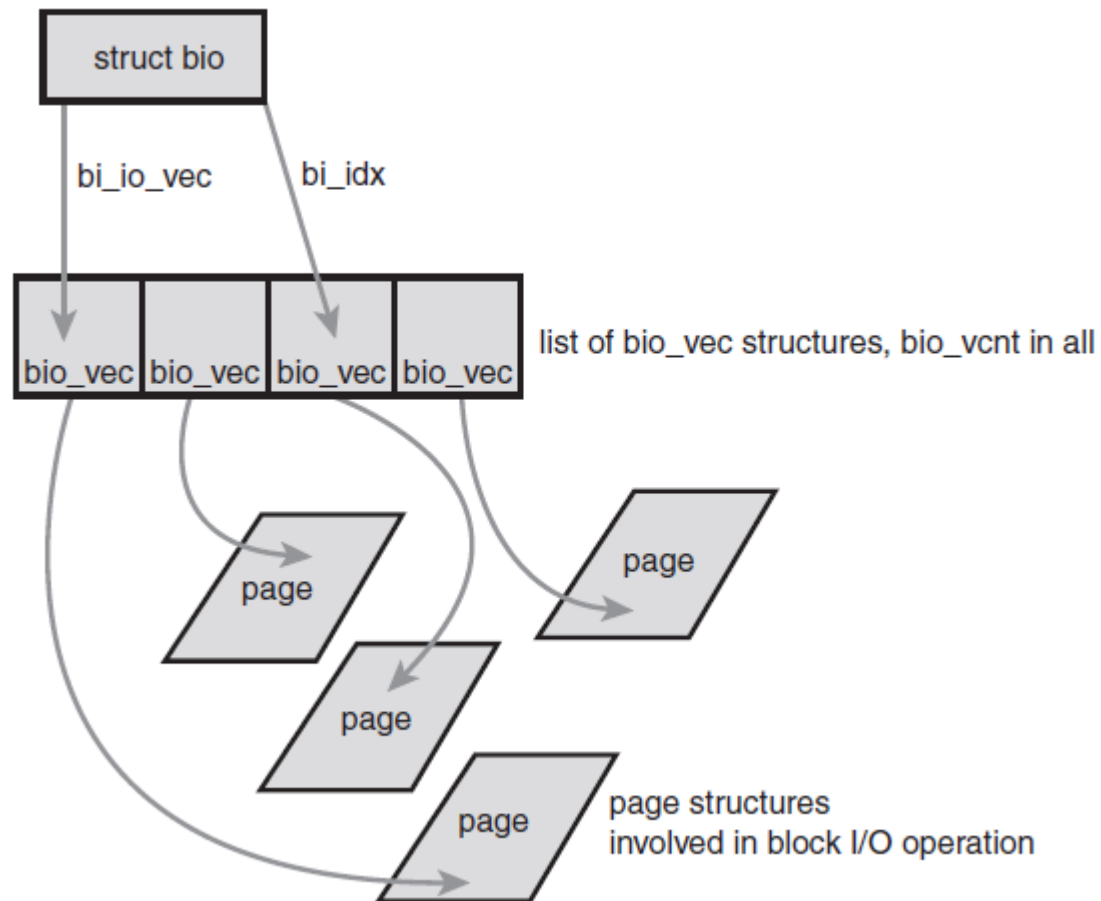
- Bufor – obiekt reprezentujący blok dysku wczytany do pamięci np. po odczycie bądź podczas zapisu.
- Deskryptor bufora (*buffer head*) – zawiera wszystkie niezbędne dla Kernela informacje dotyczące bufora. Deskryptor ten zrealizowany jest w postaci struktury `buffer_head` zdefiniowanej w `<linux/buffer_head.h>`.

```
struct buffer_head{
    unsigned long b_state;                //buffer stat flags
    struct buffer_head *b_this_page;      //list of page's buffers
    struct page *b_page;                  //associated page
    sector_t b_blocknr;                   //starting block number
    size_t b_size;                        //size of mapping
    char *b_data;                         //pointer to data within the page
    struct block_device *b_bdev;           //associated block device
    bh_end_io_t *b_end_io;                 //I/O completion
    void *b_private;                      //reservet fir b_end_io
    struct list_head b_assoc_buffers;      //associated mappings
    struct address_space *b_assoc_map;     //associated address space
    atomic_t b_count;                     //use count
}
```

Struktura bio (1)

- Reprezentuje operacje blokowe I/O, które są aktywne w postaci listy segmentów (segment – fragment bufora, który jest ciągły w pamięci).
- Dzięki takiej organizacji buforów nie muszą zajmować jednego ciągłego miejsca w pamięci.
- Opis bufora w postaci segmentów pozwala Kernelowi wykonywać operacje blokowe I/O na pojedynczym buforze rozproszonym w pamięci – *scatter-gather I/O*
- Zdefiniowana w `<linux/bio.h>`

Struktura bio (2)



Struktura bio (3)

```
struct bio{
    sector_t bi_sector;           //associated sector on disk
    struct bio *bi_next;          //list of requests
    struct block_device *bi_bdev; //associated block device
    unsigned long bi_flags;       //status and command flags
    unsigned long bi_rw;          //read or write
    unsigned short bi_vcnt;       //number of bio_vecs off
    unsigned short bi_idx;        //current index in bi_io_vec
    unsigned short bi_phys_segments; //number of segments
    unsigned int bi_size;         //I/O count
    unsigned int bi_seg_front_size; //size of first segment
    unsigned int bi_seg_back_size; //size of last segment
    unsigned int bi_max_vecs;     //maximum bio_vecs possible
    unsigned int bi_comp_cpu;     //completion CPU
    atomic_t bi_cnt;              //usage counter
    struct bio_vec *bi_io_vec;    //bio_vec list
    bio_end_io_t *bi_end_io;      //I/O completion method
    void *bi_private;             //owner-private method
    bio_destructor_t *bi_desctructor; //desctructor method
    struct bio_vec bi_inline_vecs[0]; //inline bio vectors
}
```

bi_io_vec & bio_vec

- *bio_vec* – traktowane jako wektor w postaci <page, offset, len> opisujący segment
- Bufor opisywany jest przez tablicę *bi_io_vec* wektorów *bio_vec* (zdefiniowanych w <linux/bio.h>)

```
struct bio_vec{  
    struct page *bv_page; //pointer to the physical page  
    unsigned int bv_len;   //length in bytes of this buffer  
    unsigned int bv_offset; //byte offset within the page  
}
```

- Każdy żądanie dostępu do bloku I/O reprezentowane jest w postaci struktury *bio*. Każde żądanie składa się z jednego bądź większej ilości bloków, które zawarte są w tablicy struktur *bio_vec*.

buffer_head vs bio

buffer_head	bio
reprezentuje pojedynczy bufor opisujący pojedynczy blok dysku	reprezentuje operację I/O, która może być wykonywana na jednej bądź większej ilości stron pamięci
nieefektywny podziału żądania w porcje wielkości bloku w celu obsługi (dzielimy, obsługujemy, scalamy)	pozwała na opis bloków nieciągłych, dzięki czemu nie wymaga zbędnego podziału operacji I/O
bazowanie na bezpośrednich wskaźnikach	możliwość łatwej prezentacji „dalekiej pamięci” dzięki bazowaniu na stonach fizycznych
-	możliwość reprezentacji zwykłej operacji opartej na stronie I/O oraz bezpośredniego dostępu I/O (operacji nieprzechodzących przez page cache)
-	łatwa obsługa operacji blokowego scatter-gather (vektorowego) I/O dla danych pochodzących z różnych stron fizycznych
-	posiada jedynie minimum informacji wymaganych w celu reprezentacji operacji blokowego I/O
Zawiera informacje opisujące bufor	Nie zawiera informacji o stanie bufora

Żądania I/O

- Kolejka żądań dostępu – reprezentowana przez strukturę *request_queue* zdefiniowaną w `<linux/blkdev.h>` zawiera listę dwukierunkową żądań oraz powiązane z nimi informacje kontrolne.
- Żądania dodawane do kolejki są z poziomu kodu wyższych warstw, np. z poziomu warstwy systemu plików.
- Każde żądanie znajdujące się w kolejce jest strukturą *request*. Każde żądanie może składać się z więcej niż jednej struktur *bio* – jedno żądanie może opierać się na kilku kolejnych blokach dysku.

Kolejki żądań

- Optymalizacja wydajności poprzez przede wszystkim zmniejszenie ilości przeszukiwań dysku.
- Kernel nie obsługuje żądań w kolejności ich otrzymywania ani w momencie ich otrzymania.
- W celu optymalizacji dokonywane są operacje scalenia oraz sortowania żądań.

I/O schedulers

- Celem schedulera jest wirtualizacja zasobów pomiędzy wieloma obiektami.
- Decyduje o kolejności oraz o momencie przekazania żądania do urządzenia.
- Redukuje ilość przeszukiwań dysków w celu zwiększenia *globalnej przepustowości*.
- Scalanie żądań – w przypadku pobierania przylegających do siebie sektorów
- Sortowanie żądań – w celu minimalizacji przeszukiwań dysku

Linus Elevator

- Domyślny scheduler w wersji 2.4 kernela <block/elevator.c>
- W momencie pojawienia się nowego żądania możliwe są 4 operacje:
 - gdy w kolejce znajduje się żądanie do przyległego sektora dysku, żądania te są scalane,
 - gdy żądanie znajdujące się w kolejce jest w niej już długo, nowe żądanie jest umieszczane na końcu kolejki – w celu uniknięcia zagłodzenia starszego żądania,
 - gdy w kolejce znajdowane są żądania do bliskich sektorów nieprzylegających do siebie ani do sektorów obsługiwanych przez nowe żądanie – jest ono umieszczane pomiędzy tymi żądaniami z kolejki
 - gdy nie zachodzą powyższe reguły, żądanie umieszczane jest na końcu kolejki

Problemy Linus Elevator

- intensywność operacji wykonywanych na jednej części dysku może zagłodzić inne operacje
- „write starving reads”:
 - Żądania zapisu mogą być zatwierdzane asynchronicznie – gdy kernel uzyska dostęp do nich
 - Żądania odczytu są synchroniczne i blokują aplikację do momentu ich zatwierdzenia

Deadline I/O scheduler

- Deadline I/O scheduler (<block/deadline-iosched.c>) - Każde żądanie jest oznaczone czasem wygaśnięcia (500milisekund dla operacji odczytu, 5 sekund dla operacji zapisu). Poza posortowaną wg kolejności sektorów kolejką dodatkowa implementacja dwóch kolejek FIFO (dla odczytu oraz dla zapisu)
- Anticipatory I/O scheduler <block/as-iosched.c> – heurystycznie przewiduje operacje, które mogą zaistnieć na podstawie statystyk dla każdego z procesów
- CFQ I/O scheduler <block/cfq-iosched.c> – domyślny scheduler dla kernela 2.6 – osobna kolejka żądań dla każdego procesu, pobieranie kolejnych żądań algorytmem round-robin,
- Noop I/O scheduler <block/noop-iosched.c> – dobry dla urządzeń o losowym dostępie (np. kart pamięci flash) brak sortowania

wybór schedulera – przy starcie (elevator=as/cfq/deadline/noop)